

Avance: Implementación de Sistema Distribuido sobre VPN WireGuard

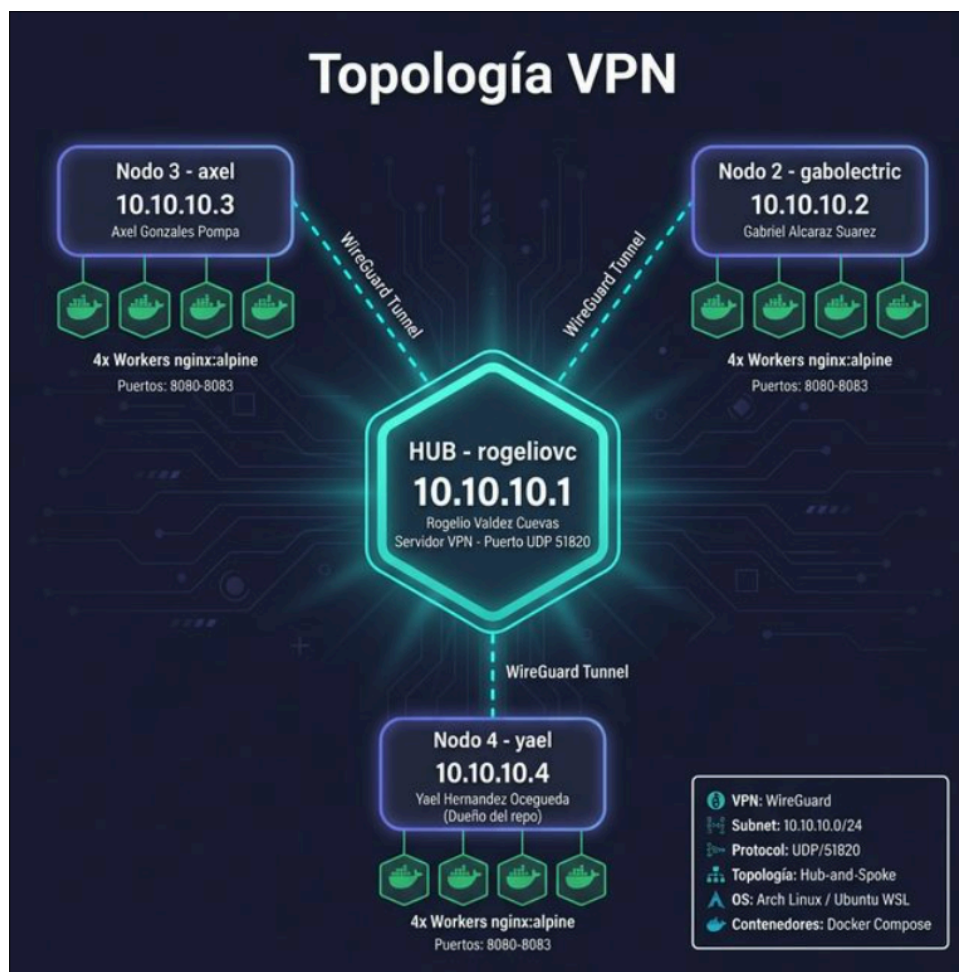
Fecha de entrega: 17 de Marzo de 2026 **Materia:** Programación Distribuida **Institución:** Universidad de Guadalajara (CUCEI)

Equipo: Rogelio, Gabriel, Axel, Yael - [laterceraeslavencida](#)

Introducción y Diagrama de la Topología VPN

Para habilitar la comunicación entre los nodos del sistema distribuido, se implementó una Red Privada Virtual (VPN) utilizando el protocolo **WireGuard**. Se optó por una topología lógica de **Hub-and-Spoke**, donde un nodo central (Hub) orquesta y enruta el tráfico entre los nodos periféricos (Spokes).

Esta arquitectura se eligió por su eficiencia frente a las restricciones de las redes residenciales (NAT estricto). Al concentrar la gestión del túnel en un solo puerto público expuesto en el Hub, los Spokes pueden conectarse dinámicamente sin necesidad de abrir puertos en sus propios enrutadores locales.



Objetivos

El presente trabajo tiene como finalidad describir, implementar y validar una arquitectura de **Cómputo Distribuido** y el entorno de red necesario para la ejecución de una carga de trabajo intensiva, demostrando la operatividad y la interconexión segura entre nodos heterogéneos. Objetivo General

Diseñar e implementar un sistema distribuido de tipo Hub-and-Spoke sobre una Red Privada Virtual (VPN) con WireGuard, utilizando la contención de Docker y el alto rendimiento de Rust, con el fin de ejecutar de manera paralela el cálculo del Conjunto de Mandelbrot.

Objetivo Particular 1 (Red): Implementar la topología lógica Hub-and-Spoke mediante el protocolo WireGuard, resolviendo los retos de conectividad en entornos NAT residenciales (Triple Salto NAT y volatilidad de la IP pública) para garantizar la interconexión segura de los nodos.

Objetivo Particular 2 (Entorno): Establecer un entorno de ejecución reproducible mediante la conteneirización con Docker, asegurando que todos los nodos Worker ejecuten el binario de Rust bajo un ambiente estandarizado.

Objetivo Particular 3 (Cómputo): Desarrollar un prototipo distribuido en Rust capaz de particionar y delegar la carga de trabajo del cálculo del Conjunto de Mandelbrot entre los nodos Worker conectados a la VPN.

Desarrollo

Descripción del Rol de Cada Nodo:

Nodo Hub (Gateway Central)

Responsable: Rogelio

Dirección IP Virtual: 10.10.10.1/24

Entorno de Ejecución: Máquina Virtual Linux (Guest) provisionada sobre un hipervisor QEMU/KVM, ejecutándose en un Host bare-metal con Arch Linux.

Funciones Principales:

- Actuar como *Gateway* central recibiendo peticiones en el puerto UDP 51820.
- Gestionar la tabla de ruteo criptográfico (Crypto-key Routing) de WireGuard.
- Proveer traducción de direcciones de red (NAT Masquerade) mediante iptables para permitir que los paquetes de los Spokes fluyan correctamente hacia su destino y retornen a la interfaz virtual wg0.
- Gestionar la tabla de ruteo criptográfico (Crypto-key Routing) de WireGuard.
- Proveer traducción de direcciones de red (NAT Masquerade) mediante iptables para permitir que los paquetes de los Spokes fluyan correctamente hacia su destino y retornen a la interfaz virtual wg0.

Nodos Spoke (Workers Distribuidos)

Responsables: Gabriel, Axel, Yael

Direcciones IP Virtuales: 10.10.10.2/24, 10.10.10.3/24, 10.10.10.4/24

Funciones Principales:

- Establecer la conexión saliente hacia el Endpoint público del Hub.
- Mantener el estado de la conexión mediante el envío de señales de vida (PersistentKeepalive = 25) para evitar el cierre de puertos por parte de los firewalls de sus respectivos ISPs.

Decisiones Técnicas y Resolución de Problemas (Troubleshooting)

La puesta en marcha de la topología enfrentó diversos retos de infraestructura, los cuales fueron documentados y resueltos mediante ingeniería de redes:

Superación del Triple Salto NAT

Las conexiones residenciales no permiten tráfico entrante no solicitado. Para que los Spokes alcancen al Hub, se diseñó una canalización a través de tres capas:

1. **Capa Perimetral (ISP):** En el módem Huawei HG8145V5V3 (Gateway de internet del Hub), se implementó una regla de **Port Forwarding** estricta, redirigiendo el tráfico entrante del puerto UDP 51820 hacia la IP local del Host físico (192.168.1.67).
2. **Capa del Host (Arch Linux):** Para comunicar el hardware físico con el entorno virtualizado, se habilitó el reenvío de paquetes en el kernel (sysctl -w net.ipv4.ip_forward=1). Posteriormente, se inyectó una regla DNAT en la cadena PREROUTING de iptables para desviar los paquetes hacia la subred de la máquina virtual.
3. **Capa Virtual (Guest):** En la configuración de WireGuard del Hub, se automatizó la creación de reglas POSTROUTING -j MASQUERADE al levantar la interfaz (PostUp), permitiendo la correcta respuesta a los clientes.

Volatilidad de la IP Pública (PPPoE)

Durante las pruebas de conectividad con el nodo de Gabriel, se documentó una caída del servicio. El análisis de diagnóstico reveló que la conexión WAN del ISP residencial utiliza sesiones PPPoE dinámicas. La IP pública del Hub cambió repentinamente de 189.143.175.140 a 187.201.14.25.

Decisión técnica: Al no contar con una IP Estática comercial, se instruyó a los nodos Spoke actualizar manualmente la variable Endpoint en sus archivos de configuración. Se dejó el campo Endpoint vacío en la configuración del Hub para aprovechar el comportamiento de *Roaming* de WireGuard, permitiendo que el Hub actualice dinámicamente la ubicación de los Spokes sin importar si cambian de red. (Queda pendiente el usar el DNS de infinitum)

Evidencia de Configuración y Conectividad VPN

Generación de Llaves WireGuard

WireGuard opera bajo un esquema de **Cripto-Ruteo** (Crypto-key Routing). La identidad del nodo y su dirección IP están criptográficamente vinculadas. Las llaves se generaron utilizando la curva elíptica Curve25519 con el siguiente comando:

Bash

```
wg genkey | tee privatekey | wg pubkey > publickey
```

```
gaboLectric# cat /etc/wireguard/publickey  
MzeqrUWkSBuHEs1jG5u6io665zzAiV0b0KDexkbMji4=  
gaboLectric# |
```

```
axel_@Pomposauria:~$ sudo cat /etc/wireguard/publickey  
1jEGvVWHzCxcF+dxDvfU8RgPisgP5K+VCe2bvh9qqyc=  
axel_@Pomposauria:~$ █
```

Estado de las Interfaces de Red

Salida de `wg show` en el nodo Hub: La siguiente evidencia demuestra el establecimiento exitoso del túnel (Handshake) y la transferencia bidireccional de bytes, confirmando que la criptografía asimétrica y el túnel UDP están operando.

```
(rogeliovm) localhost:2222 — Konsole  
Nueva pestaña  Dividir vista  Copiar  Pegar  Buscar...  ≡  
[rogeliovm@archlinux proyecto-distribuidos]$ sudo wg show  
interface: wg0  
  public key: NK3tGMGpPvjhFjbrNEGtinhMn/lEFp5V0roL4N+iNBw=  
  private key: (hidden)  
  listening port: 51820  
  
peer: MzeqrUWkSBuHEs1jG5u6io665zzAiV0b0KDexkbMji4=  
  endpoint: 177.241.54.83:10558  
  allowed ips: 10.10.10.2/32  
  latest handshake: 7 seconds ago  
  transfer: 48.06 KiB received, 40.58 KiB sent  
  
peer: 1jEGvVWHzCxcF+dxDvfU8RgPisgP5K+VCe2bvh9qqyc=  
  endpoint: 187.188.64.60:54196  
  allowed ips: 10.10.10.3/32  
  latest handshake: 17 seconds ago  
  transfer: 180 B received, 92 B sent  
  
peer: 100g58MT4lS8z4Ixp++0eYWvcuV0p/d44i1uDkDsEY=  
  endpoint: 177.231.15.207:14685  
  allowed ips: 10.10.10.4/32  
  latest handshake: 5 minutes, 19 seconds ago  
  transfer: 16.54 KiB received, 13.64 KiB sent  
[rogeliovm@archlinux proyecto-distribuidos]$ █
```

Pruebas de Red Transversales

Ping entre todos los nodos:

```
gaboLectric# ping 10.10.10.4
PING 10.10.10.4 (10.10.10.4) 56(84) bytes of data.
64 bytes from 10.10.10.4: icmp_seq=1 ttl=63 time=52.2 ms
64 bytes from 10.10.10.4: icmp_seq=2 ttl=63 time=53.5 ms
64 bytes from 10.10.10.4: icmp_seq=3 ttl=63 time=58.7 ms
64 bytes from 10.10.10.4: icmp_seq=4 ttl=63 time=57.3 ms
64 bytes from 10.10.10.4: icmp_seq=5 ttl=63 time=50.3 ms
64 bytes from 10.10.10.4: icmp_seq=6 ttl=63 time=200 ms

--- 10.10.10.4 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5008ms
rtt min/avg/max/mdev = 50.271/78.702/200.186/54.404 ms
gaboLectric#
```

```
yael@DESKTOP-1B05468:/proyecto2/los-dockerinos/DOCKER$ ping 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=2 ttl=63 time=94.1 ms
64 bytes from 10.10.10.2: icmp_seq=3 ttl=63 time=59.9 ms
64 bytes from 10.10.10.2: icmp_seq=4 ttl=63 time=62.4 ms
64 bytes from 10.10.10.2: icmp_seq=5 ttl=63 time=55.3 ms
64 bytes from 10.10.10.2: icmp_seq=6 ttl=63 time=68.6 ms
64 bytes from 10.10.10.2: icmp_seq=7 ttl=63 time=54.7 ms
64 bytes from 10.10.10.2: icmp_seq=8 ttl=63 time=79.9 ms
64 bytes from 10.10.10.2: icmp_seq=9 ttl=63 time=62.4 ms
64 bytes from 10.10.10.2: icmp_seq=10 ttl=63 time=75.1 ms
64 bytes from 10.10.10.2: icmp_seq=11 ttl=63 time=62.7 ms
^C
--- 10.10.10.2 ping statistics ---
11 packets transmitted, 10 received, 9.09091% packet loss, time 10001ms
rtt min/avg/max/mdev = 54.684/67.503/94.132/11.705 ms
yael@DESKTOP-1B05468:/proyecto2/los-dockerinos/DOCKER$
```

Prueba completa de iperf3:

```
[rogeliovm@archlinux ~]$ iperf3 -s
-----
Server listening on 5201 (test #1)
-----
Accepted connection from 10.10.10.3, port 57364
[ 5] local 10.10.10.1 port 5201 connected to 10.10.10.3 port 57380
[ ID] Interval           Transfer     Bitrate
[ 5] 0.00-1.00 sec      0.00 Bytes  0.00 bits/sec
[ 5] 1.00-2.00 sec      640 KBytes  5.25 Mbits/sec
[ 5] 2.00-3.00 sec      896 KBytes  7.34 Mbits/sec
[ 5] 3.00-4.00 sec      0.00 Bytes  0.00 bits/sec
[ 5] 4.00-5.00 sec      1.75 MBytes 14.7 Mbits/sec
[ 5] 5.00-6.00 sec      2.25 MBytes 18.9 Mbits/sec
[ 5] 6.00-7.00 sec      2.00 MBytes 16.8 Mbits/sec
[ 5] 7.00-8.00 sec      2.38 MBytes 19.9 Mbits/sec
[ 5] 8.00-9.00 sec      2.50 MBytes 21.0 Mbits/sec
[ 5] 9.00-9.85 sec      2.12 MBytes 21.0 Mbits/sec
-----
[ ID] Interval           Transfer     Bitrate
[ 5] 0.00-9.85 sec      14.5 MBytes 12.3 Mbits/sec
receiver
```



```

axel_@Pomposauria:~$ iperf3 -c 10.10.10.1
Connecting to host 10.10.10.1, port 5201
[ 5] local 10.10.10.3 port 57380 connected to 10.10.10.1 port 5201
[ ID] Interval           Transfer     Bitrate      Retr   Cwnd
[ 5]  0.00-1.00       sec    768 KBytes  6.27 Mbits/sec    1   188 KBytes
[ 5]  1.00-2.00       sec   2.25 MBytes 18.9 Mbits/sec    0   754 KBytes
[ 5]  2.00-3.00       sec    0.00 Bytes  0.00 bits/sec    1   764 KBytes
[ 5]  3.00-4.00       sec   1.00 MBytes  8.40 Mbits/sec    6   550 KBytes
[ 5]  4.00-5.00       sec   1.88 MBytes 15.7 Mbits/sec   16   563 KBytes
[ 5]  5.00-6.00       sec   1.88 MBytes 15.7 Mbits/sec   58   304 KBytes
[ 5]  6.00-7.00       sec   2.00 MBytes 16.8 Mbits/sec    0   328 KBytes
[ 5]  7.00-8.00       sec   1.88 MBytes 15.7 Mbits/sec    0   344 KBytes
[ 5]  8.00-9.00       sec   1.88 MBytes 15.7 Mbits/sec    0   351 KBytes
[ 5]  9.00-10.05      sec   2.75 MBytes 22.1 Mbits/sec    0   352 KBytes
- - - - -
[ ID] Interval           Transfer     Bitrate      Retr
[ 5]  0.00-10.05      sec  16.2 MBytes 13.6 Mbits/sec   82
[ 5]  0.00-9.85       sec  14.5 MBytes 12.3 Mbits/sec
sender
receiver

iperf Done.
axel_@Pomposauria:~$

```

Instalación de Entorno de Contenedores

Dado que los nodos Spoke y Hub operan bajo distintas distribuciones y configuraciones de Linux, se decidió contenerizar el ambiente de ejecución. Docker asegura que el binario de Rust interactúe con la misma versión de la biblioteca estándar (glibc) y el mismo entorno de red.

```

gaboElectric# sudo pacman -S docker-compose
warning: docker-compose-5.0.2-1 is up to date -- reinstalling
resolving dependencies...
looking for conflicting packages...

Packages (1) docker-compose-5.0.2-1

Total Installed Size: 26.90 MiB
Net Upgrade Size:      0.00 MiB

:: Proceed with installation? [Y/n] Y
(1/1) checking keys in keyring
(1/1) checking package integrity
(1/1) loading package files
(1/1) checking for file conflicts
(1/1) checking available disk space
:: Processing package changes...
(1/1) reinstalling docker-compose
:: Running post-transaction hooks...
(1/1) Arming ConditionNeedsUpdate...
gaboElectric#

```

Evidencia de instalación (docker ps):

Hub:

```

[rogeliovm@archlinux proyecto-distribuidos]$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
d53029e8da96   nginx:alpine   "/docker-entrypoint..." 42 minutes ago Up 42 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   proyecto-distribuidos-worker-3
159b26ba03e8   nginx:alpine   "/docker-entrypoint..." 42 minutes ago Up 42 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   proyecto-distribuidos-worker-4
8ca33c36ba7e   nginx:alpine   "/docker-entrypoint..." 42 minutes ago Up 42 minutes 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp   proyecto-distribuidos-worker-2
22188c35597d   nginx:alpine   "/docker-entrypoint..." 42 minutes ago Up 42 minutes 0.0.0.0:8083->80/tcp, [::]:8083->80/tcp   proyecto-distribuidos-worker-1
[rogeliovm@archlinux proyecto-distribuidos]$

```

Peer 1:

```
gaboLectric# sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
772b6ed297db   nginx:alpine  "/docker-entrypoint..." 38 seconds ago Up 37 seconds 10.10.10.2:8080->80/tcp   docker-worker-1
5c1b60ff98bc   nginx:alpine  "/docker-entrypoint..." 38 seconds ago Up 37 seconds 10.10.10.2:8081->80/tcp   docker-worker-3
b68f396d47c5   nginx:alpine  "/docker-entrypoint..." 38 seconds ago Up 37 seconds 10.10.10.2:8082->80/tcp   docker-worker-4
3d04a4efd82c   nginx:alpine  "/docker-entrypoint..." 38 seconds ago Up 37 seconds 10.10.10.2:8083->80/tcp   docker-worker-2
gaboLectric# |
```

Peer 2:

```
axel_@Pomposauria:~/los-dockerinos/DOCKER$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
2fce7c90f07c   nginx:alpine  "/docker-entrypoint..." 13 minutes ago Up 15 seconds 10.10.10.3:8083->80/   docker_worker_1
tcp
7f69deb72f9a   nginx:alpine  "/docker-entrypoint..." 13 minutes ago Up 15 seconds 10.10.10.3:8080->80/   docker_worker_4
tcp
b7869dcd4db    nginx:alpine  "/docker-entrypoint..." 13 minutes ago Up 15 seconds 10.10.10.3:8081->80/   docker_worker_2
tcp
1ca0d7b00f3c   nginx:alpine  "/docker-entrypoint..." 13 minutes ago Up 15 seconds 10.10.10.3:8082->80/   docker_worker_3
tcp
```

Peer 3:

```
yael@DESKTOP-1B05468:/proyecto2/los-dockerinos/DOCKER$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
0df5f71d5f07   nginx:alpine  "/docker-entrypoint..." 14 minutes ago Up 14 minutes 10.10.10.4:8080->80/tcp   docker_worker_3
efcca774bf6b   nginx:alpine  "/docker-entrypoint..." 14 minutes ago Up 14 minutes 10.10.10.4:8083->80/tcp   docker_worker_4
7da125cc8e5c   nginx:alpine  "/docker-entrypoint..." 14 minutes ago Up 14 minutes 10.10.10.4:8082->80/tcp   docker_worker_2
44976e660b8c   nginx:alpine  "/docker-entrypoint..." 14 minutes ago Up 14 minutes 10.10.10.4:8081->80/tcp   docker_worker_1
3dff97408be1   proyecto-distribuido_worker  "/target/release/wo..." 2 days ago    Up About an hour          proyecto-distribuido_worker_3
c66e1481381f   proyecto-distribuido_worker  "/target/release/wo..." 2 days ago    Up About an hour          proyecto-distribuido_worker_4
60b20393d6f0   proyecto-distribuido_worker  "/target/release/wo..." 2 days ago    Up About an hour          proyecto-distribuido_worker_1
95ec298cba90   proyecto-distribuido_worker  "/target/release/wo..." 2 days ago    Up About an hour          proyecto-distribuido_worker_2
yael@DESKTOP-1B05468:/proyecto2/los-dockerinos/DOCKER$ |
```

Definición del archivo docker-compose.yml base: Este archivo define el servicio *worker*, enlazándolo a la red del host para aprovechar directamente la interfaz wg0 de la VPN.

```
gaboLectric# cat docker-compose.yml
version: '3.8'

services:
  worker:
    image: nginx:alpine
    deploy:
      replicas: 4
    ports:
      - "10.10.10.2:8080-8083:80"
    restart: unless-stopped
gaboLectric# |
```

Sistema Distribuido para el Cálculo del Conjunto de Mandelbrot

Definición y Justificación del Problema

El objetivo de este proyecto es el cálculo y renderizado eficiente del **Conjunto de Mandelbrot**, un fractal cuya generación requiere evaluar cada punto del plano complejo mediante múltiples iteraciones. Al trabajar con imágenes de alta resolución, este proceso demanda millones de cálculos independientes, lo que convierte a la ejecución secuencial en un cuello de botella crítico.

Para resolver esta limitación, se implementó un sistema de cómputo distribuido basado en los siguientes pilares:

- **Paralelismo masivo:** El plano complejo se divide en regiones independientes que son procesadas simultáneamente por diversos nodos, reduciendo el tiempo total de ejecución de forma proporcional al número de trabajadores disponibles.
- **Alto rendimiento con Rust:** Se seleccionó Rust por su eficiencia comparable a C/C++ y su modelo de seguridad de memoria. Esto permite ejecutar cálculos matemáticos intensivos de forma concurrente sin las penalizaciones de rendimiento de un recolector de basura (*garbage collector*).
- **Contenerización con Docker:** El uso de contenedores garantiza la homogeneidad del entorno de software en todos los nodos, eliminando conflictos de dependencias y agilizando el despliegue de nuevos *workers*.
- **Seguridad mediante VPN:** La comunicación se blindó a través de una red privada con **WireGuard**, asegurando que tanto las tareas como los resultados viajen cifrados y bajo control.

Arquitectura del Sistema

El sistema adopta una arquitectura **Master-Worker**, donde un nodo central coordina la carga de trabajo ejecutada por múltiples nodos secundarios.

Coordinador (Nodo Master)

Actuando como el centro neurálgico, el coordinador se implementó como un servicio HTTP utilizando el framework **Axum** sobre el runtime asíncrono **Tokio**. Sus responsabilidades clave incluyen:

1. Fraccionar la imagen en regiones de trabajo.
2. Gestionar la asignación de tareas mediante el endpoint `/get_task`.
3. Consolidar los resultados recibidos en `/submit_result`.
4. Ensamblar la imagen final tras completar el procesamiento.

Para garantizar la integridad de los datos en un entorno concurrente, el estado global se gestiona mediante punteros compartidos (`Arc<Mutex<T>>`), evitando condiciones de carrera entre solicitudes simultáneas.

Workers (Nodos de Cómputo)

Los *workers* son clientes HTTP que consumen tareas del coordinador utilizando la biblioteca **Reqwest**. Su ciclo operativo es el siguiente:

- **Solicitud:** Obtienen una región asignada del coordinador.
- **Cómputo:** Ejecutan el algoritmo del fractal sobre dicha región.
- **Transformación:** Convierten los resultados en un mapa de píxeles en escala de grises.
- **Retorno:** Envían los datos procesados al maestro.

Además, cuentan con mecanismos de **tolerancia a fallos**: si el coordinador no responde, el *worker* activa un ciclo de reintento automático cada cinco segundos.

Modelado de Datos y Comunicación

La interoperabilidad entre nodos se logra mediante mensajes JSON serializados con la biblioteca **Serde**. El flujo de información se estructura en tres entidades principales:

- **MandelbrotParams:** Define las dimensiones globales, los límites del plano complejo y el límite de iteraciones.
- **Task / ImageRegion:** Representa la unidad mínima de trabajo, incluyendo su identificador único y las coordenadas específicas de la región.
- **TaskResult:** Contiene el ID de la tarea y del trabajador, el tiempo de procesamiento y el vector de bytes (**Vecu8**) con los píxeles calculados.

Algoritmo de Cálculo

El núcleo del sistema es la iteración de la función compleja:

$$z_{n+1} = z_n^2 + c$$

Para cada píxel, se mapea su posición al plano complejo y se itera la ecuación hasta que la magnitud de z exceda el radio de escape (2.0) o se alcance el máximo de iteraciones. El resultado se normaliza a un valor entre 0 y 255 para generar una escala de grises: los puntos pertenecientes al conjunto se representan en negro, mientras que los externos varían hacia el blanco según su velocidad de escape.

Infraestructura y Despliegue

Estrategia de Contenerización

Se empleó una estrategia de **multi-stage build** en Docker para optimizar el despliegue:

1. **Etapla de construcción:** Utiliza una imagen **rust:alpine** para compilar un binario optimizado en modo *release*.
2. **Etapla de ejecución:** Copia el binario resultante a una imagen **alpine** mínima. Esto reduce el peso de la imagen final a solo **50 MB**, acelerando la distribución en la red.

Orquestación y Red

Mediante **Docker Compose**, se facilita el escalado horizontal de los *workers*. La configuración utiliza **network_mode: host** para permitir que los contenedores se comuniquen directamente a través de la interfaz de la VPN WireGuard, la cual utiliza una topología *Hub-and-Spoke*. En pruebas reales, esta configuración mantuvo latencias inferiores a los 50 ms.

Dockers de cada nodo trabajando en conjunto:

```
Attaching to rust_worker_1, rust_worker_2, rust_worker_3, rust_worker_4
rust_worker_2 | * Modo Worker
rust_worker_2 | * Conectado a: 10.10.10.1:3000
rust_worker_2 | * Worker ID: worker-gabolectric-2
rust_worker_2 | * Iniciando Worker worker-gabolectric-2... Conectando al Coordinador en 10.10.10.1:3000
rust_worker_4 | * Modo Worker
rust_worker_4 | * Conectado a: 10.10.10.1:3000
rust_worker_4 | * Worker ID: worker-gabolectric-4
rust_worker_4 | * Iniciando Worker worker-gabolectric-4... Conectando al Coordinador en 10.10.10.1:3000
rust_worker_3 | * Modo Worker
rust_worker_3 | * Conectado a: 10.10.10.1:3000
rust_worker_3 | * Worker ID: worker-gabolectric-3
rust_worker_3 | * Iniciando Worker worker-gabolectric-3... Conectando al Coordinador en 10.10.10.1:3000
rust_worker_1 | * Modo Worker
rust_worker_1 | * Conectado a: 10.10.10.1:3000
rust_worker_1 | * Worker ID: worker-gabolectric-1
rust_worker_1 | * Iniciando Worker worker-gabolectric-1... Conectando al Coordinador en 10.10.10.1:3000

rust_worker_3 | Worker worker-gabolectric-3: Recibió tarea 10 para región [0,800] x [533,370]
rust_worker_4 | Worker worker-gabolectric-4: Recibió tarea 16 para región [0,800] x [443,516]
rust_worker_2 | Worker worker-gabolectric-2: Recibió tarea 13 para región [0,800] x [444,481]
rust_worker_1 | Worker worker-gabolectric-1: Recibió tarea 12 para región [0,800] x [487,444]

rust_worker_2 | Worker worker-gabolectric-2: Cálculo completado en 12ms, 29600 pixeles generados
rust_worker_4 | Worker worker-gabolectric-4: Cálculo completado en 5ms, 29600 pixeles generados
rust_worker_3 | Worker worker-gabolectric-3: Cálculo completado en 25ms, 29600 pixeles generados
rust_worker_1 | Worker worker-gabolectric-1: Cálculo completado en 21ms, 29600 pixeles generados
rust_worker_2 | Worker worker-gabolectric-2: Resultado enviado al coordinador
rust_worker_4 | Worker worker-gabolectric-4: Resultado enviado al coordinador
rust_worker_3 | Worker worker-gabolectric-3: Resultado enviado al coordinador
rust_worker_1 | Worker worker-gabolectric-1: Resultado enviado al coordinador

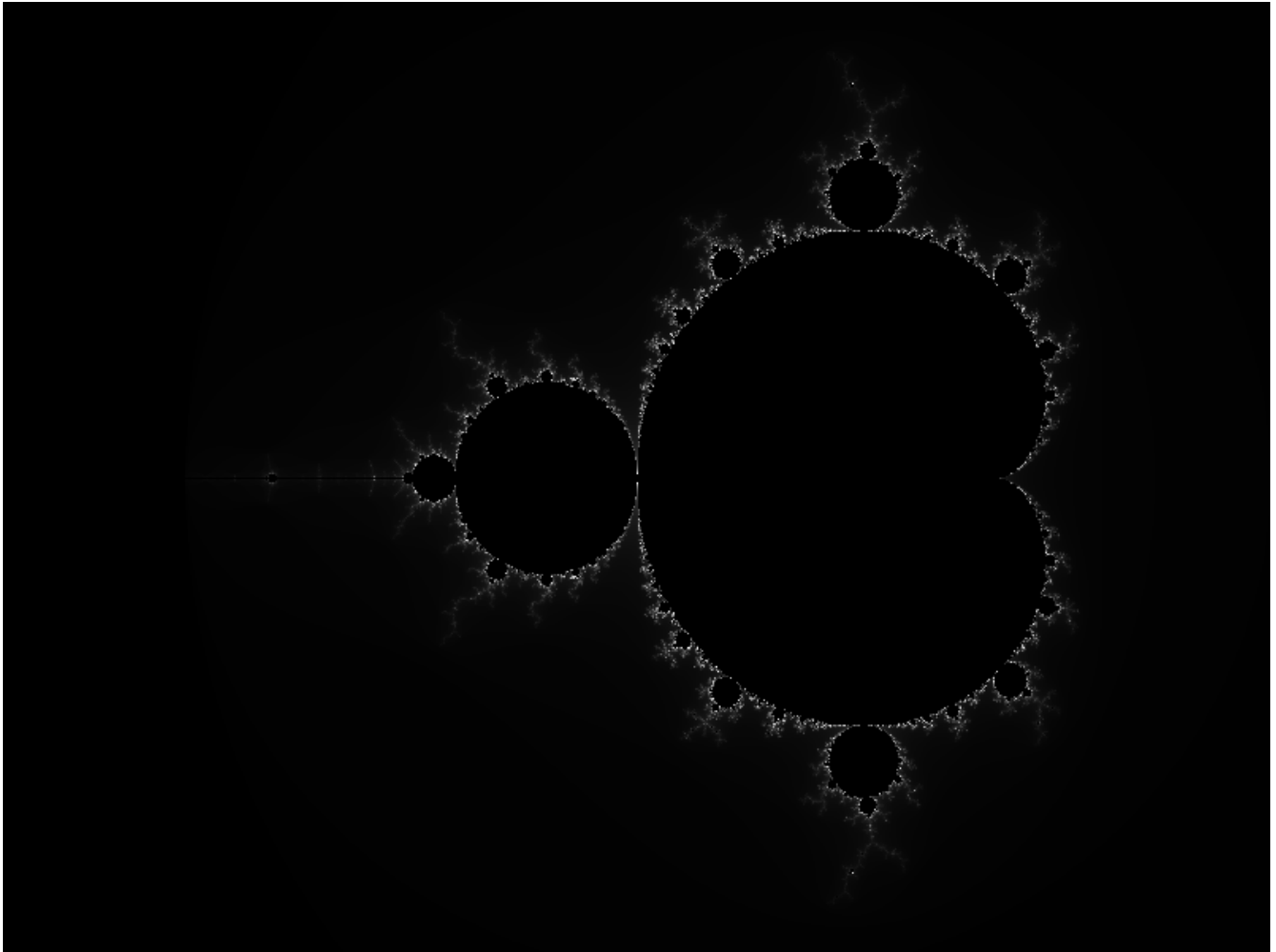
rust_worker_3 | Worker worker-axel-3: Recibió tarea 7 para región [0,800] x [222,259]
rust_worker_4 | Worker worker-axel-4: Recibió tarea 6 para región [0,800] x [185,222]
rust_worker_1 | Worker worker-axel-1: Recibió tarea 5 para región [0,800] x [148,185]
rust_worker_2 | Worker worker-axel-2: Recibió tarea 8 para región [0,800] x [259,296]
rust_worker_4 | Worker worker-axel-4: Cálculo completado en 13ms, 29600 pixeles generados
rust_worker_3 | Worker worker-axel-3: Cálculo completado en 17ms, 29600 pixeles generados
rust_worker_1 | Worker worker-axel-1: Cálculo completado en 27ms, 29600 pixeles generados
rust_worker_2 | Worker worker-axel-2: Resultado enviado al coordinador
rust_worker_4 | Worker worker-axel-4: Resultado enviado al coordinador
rust_worker_3 | Worker worker-axel-3: Resultado enviado al coordinador
rust_worker_1 | Worker worker-axel-1: Resultado enviado al coordinador
rust_worker_2 | Worker worker-axel-2: Recibió tarea 9 para región [0,800] x [296,333]
rust_worker_4 | Worker worker-axel-4: Recibió tarea 10 para región [0,800] x [333,370]
rust_worker_3 | Worker worker-axel-3: Recibió tarea 8 para región [0,800] x [259,296]
rust_worker_1 | Worker worker-axel-1: Recibió tarea 11 para región [0,800] x [370,487]
rust_worker_2 | Worker worker-axel-2: Cálculo completado en 19ms, 29600 pixeles generados
rust_worker_4 | Worker worker-axel-4: Cálculo completado en 24ms, 29600 pixeles generados
rust_worker_3 | Worker worker-axel-3: Cálculo completado en 27ms, 29600 pixeles generados
rust_worker_1 | Worker worker-axel-1: Cálculo completado en 29ms, 29600 pixeles generados
rust_worker_2 | Worker worker-axel-2: Resultado enviado al coordinador
rust_worker_4 | Worker worker-axel-4: Resultado enviado al coordinador
rust_worker_3 | Worker worker-axel-3: Resultado enviado al coordinador
rust_worker_1 | Worker worker-axel-1: Resultado enviado al coordinador
rust_worker_2 | Worker worker-axel-2: Resultado enviado al coordinador
rust_worker_4 | Worker worker-axel-4: Resultado enviado al coordinador

rogelio@archlinux:~$ cd rust && ./target/release/mandelbrot_distribuido coordinador
* Modo Coordinador
* Iniciando Coordinador...
* Coordinador: Generadas 16 tareas para imagen 800x600
* Coordinador: escuchando en 0.0.0.0:3000
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 16 (región [0,800] x [555,600])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 15 (región [0,800] x [518,555])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 14 (región [0,800] x [481,518])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 13 (región [0,800] x [444,481])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 12 (región [0,800] x [407,444])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 11 (región [0,800] x [370,407])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 10 (región [0,800] x [333,370])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 9 (región [0,800] x [296,333])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 8 (región [0,800] x [259,296])
* Coordinador: Recibida solicitud de tarea 12 del worker worker-rogelio-1 (18ms, 29600 pixeles)
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 7 (región [0,800] x [222,259])
* Coordinador: Recibido resultado de tarea 15 del worker worker-gabolectric-1 (0ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 13 del worker worker-gabolectric-3 (7ms, 29600 pixeles)
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 6 (región [0,800] x [185,222])
* Coordinador: Recibido resultado de tarea 7 del worker worker-rogelio-3 (36ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 6 del worker worker-rogelio-4 (17ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 14 del worker worker-gabolectric-2 (7ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 10 del worker worker-gabolectric-4 (0ms, 36800 pixeles)
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 5 (región [0,800] x [148,185])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 4 (región [0,800] x [111,148])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 3 (región [0,800] x [74,111])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 2 (región [0,800] x [37,74])
* Coordinador: Recibida solicitud de tarea
* Coordinador: Asignando tarea 1 (región [0,800] x [0,37])

* Coordinador: Recibido resultado de tarea 11 del worker worker-axel-4 (19ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 5 del worker worker-rogelio-2 (8ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 8 del worker worker-axel-1 (27ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 10 del worker worker-axel-2 (24ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 9 del worker worker-axel-3 (29ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 1 del worker worker-yael-3 (0ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 2 del worker worker-yael-1 (0ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 3 del worker worker-yael-4 (1ms, 29600 pixeles)
* Coordinador: Recibido resultado de tarea 4 del worker worker-yael-2 (2ms, 29600 pixeles)
* Coordinador: ¡Todas las 16 tareas completadas! Iniciando ensamblaje...
* Coordinador: Ensamblando imagen final...
* Coordinador: Imagen final ensamblada:
- Total pixeles: 480000
- Tiempo total de cómputo: 187ms
- Tareas procesadas: 16/16
- Tiempo promedio por tarea: 11ms
```

Resultados y Resolución de Problemas

Tras la recepción de todos los fragmentos, el coordinador ordena las tareas por su ID y reconstruye la matriz completa para generar un archivo **.png** mediante la biblioteca **image**.



Durante el desarrollo se resolvieron desafíos críticos, tales como:

- **Almacenamiento:** Se amplió la capacidad de la VM de 8 GB a 20 GB tras detectar que la compilación de Rust agotaba el espacio en disco.
- **Resolución DNS:** Se mitigaron errores de red en imágenes Alpine mediante el uso de binarios estáticos y la optimización de dependencias externas.

Análisis de Riesgos

- **Inestabilidad de Red:** Las conexiones domésticas introducen jitter que puede afectar la sincronización.
- **SPOF (Punto Único de Falla):** La dependencia de un único coordinador central implica que su caída detiene todo el sistema.
- **Heterogeneidad:** El uso de Docker estandarizado fue vital para neutralizar las diferencias entre arquitecturas (WSL, ARM, Linux nativo).

Conclusión Integral del Sistema

La presente investigación aplicada y su posterior desarrollo demuestran la viabilidad técnica de orquestar soluciones de cómputo de alto rendimiento (HPC) sobre infraestructuras de red de grado consumidor.

Desde la perspectiva de red, el uso de WireGuard probó ser una solución elegante y moderna frente a los paradigmas tradicionales de VPN (como OpenVPN o IPsec). La topología Hub-and-Spoke, apoyada por ingeniería de ruteo (iptables y NAT Masquerading), superó exitosamente las limitaciones de las ISP residenciales en México, tales como el triple salto NAT y las IPs dinámicas, manteniendo un túnel UDP seguro con baja latencia.

Desde la perspectiva de la ingeniería de software, la selección de Rust permitió un manejo de concurrencia seguro, previniendo fugas de memoria y bloqueos cruzados (deadlocks) mediante su sistema de propiedad y tipos (Arc y Mutex). La API HTTP descentralizada distribuyó eficazmente el cálculo algorítmico del Conjunto de Mandelbrot, logrando ensamblar imágenes de alta densidad procesando casi medio millón de píxeles en fracciones de segundo.

Finalmente, la integración de Docker estandarizó los entornos de ejecución, erradicando el clásico problema de "funciona en mi máquina". La contenedorización eficiente permitió escalar horizontalmente la capacidad de procesamiento de la red con la simple ejecución de un comando de orquestación local (docker-compose up). Este proyecto no solo valida un modelo de sistema distribuido funcional, sino que establece un marco metodológico reproducible para futuras implementaciones de cómputo en la nube descentralizado o despliegues Edge Computing.